curso-js

Índice

1. curso-js > caracteristicas de JS
2. curso-js > variables
3. curso-js > Funciones
4. curso-js > Parámetros
5. curso-js > Objetos en js
6. curso-js > desestructuración de objetos
7. curso-js > arreglos
8. curso-js > operador coalescencia nula
9. curso-js > funciones y this
10. curso-js > inmutabilidad en funciones que manejan objetos
11. curso-js > programacion funcional

Nombres de entidades

PascalCase
camelCase
snake_case
kebab-case

JS

JavaScript (JS) es un lenguaje de programación ligero, interpretado, JavaScript es un lenguaje de programación basada en prototipos, multiparadigma, de un solo hilo, dinámico, con soporte para programación orientada a objetos, imperativa y declarativa (por ejemplo programación funcional). (MDNWeb Docs)

Es soportado por todos los navegadores y también se puede usar del lado del servidor con NODE.JS. NODE es una plataforma open source para el desarrollo de scripts, servidores, web apps y herramientas de liínea de comando.

características de JS

1. Imperativo: JavaScript es un lenguaje imperativo, es decir, se ejecutan las sentencias de manera secuencial.
2. Tipado débil: En JavaScript, no se define el tipo de una variable al instanciarla. El tipo de la variable se asigna según el valor que se le asigna. Además, si se cambia el valor asignado

a la variable, esta puede cambiar su tipo de datos.

3. Interpretado: JavaScript no se compila en código máquina. En su lugar, necesita un intérprete para ejecutarse. Sin embargo, motores como Chrome V8 realizan compilaciones JIT (Just-In-Time) para mejorar el rendimiento.
4. Sencillo: JavaScript es un lenguaje fácil de aprender y utilizar, no requiere conocimientos avanzados de programación para desarrollar aplicaciones.
5. Cliente y servidor: JavaScript puede ser utilizado tanto para desarrollo de aplicaciones del lado cliente (front-end) como del lado servidor (back-end) mediante tecnologías como Node.js.
6. Orientado a objetos: Aunque no es un lenguaje puro de programación orientada a objetos como C++ o Java, JavaScript utiliza prototipos para definir objetos y permite la creación de objetos y herencia de propiedades.
7. Multiplataforma: JavaScript se ejecuta en diferentes navegadores (Google Chrome, Microsoft Edge, Safari, Opera, etc.) y sistemas operativos (Windows, Mac, Linux, etc.).
8. Compatibilidad: JavaScript es compatible con la mayoría de los navegadores web modernos y se ejecuta directamente en el navegador del usuario final.
9. Extensibilidad: Existen diferentes frameworks y bibliotecas de JavaScript que permiten ampliar las funcionalidades del lenguaje y facilitar el desarrollo de aplicaciones.
10. Evolutivo: JavaScript es un lenguaje en constante evolución, con nuevas características y mejoras constantemente agregadas a los navegadores y frameworks.

A continuación un resumen del lenguaje.

tipo de datos

El último estándar ECMAScript define nueve tipos:

Seis tipos de datos primitivos, controlados por el operador typeof

1. Undefined: typeof instance === "undefined"
2. Boolean: typeof instance === "boolean"
3. Number: typeof instance === "number"
4. String: typeof instance === "string"
5. BigInt: typeof instance === "bigint"
6. Symbol: typeof instance === "symbol"
7. Null: typeof instance === "object". Tipo primitivo especial. En JavaScript, null es un valor primitivo que representa la ausencia intencional de cualquier valor u objeto. Es decir, se utiliza para indicar que una variable no tiene ningún valor asignado de forma deliberada. Aunque null es un primitivo, curiosamente, su tipo (typeof) es "object" debido a un error histórico en JavaScript.

8. Object: typeof instance === "object". Tipo estructural especial que no es de datos pero para cualquier instancia de objeto construido que también se utiliza como estructuras de datos: new Object, new Array, new Map, new Set, new WeakMap, new WeakSet, new Date y casi todo lo hecho con la palabra clave new;
9. Function: una estructura sin datos, aunque también responde al operador typeof: typeof instance === "function". Esta simplemente es una forma abreviada para funciones, aunque cada constructor de funciones se deriva del constructor Object.

variables

Como js es un lenguaje debilmente tipado, las variables no tienen tipo asignado en la declaración, se declaran así:

```
var saludo="hola"
const x=0
let nombre=true
//tipos primitivos se asignacion por valor
int x=0
iny y=x
x=2
console.log(x) //2
console.log(y) //0

//los objetos se asignan por referencia
//declaramos un objeto
let persona={
    "nombre": "Juan",
    "edad": 30
}

//al copiar, copia la referencia
let persona2=persona;
persona2.nombre="Pedro";

console.log("persona:",persona) // persona:{ nombre: 'Pedro', edad: 30 }
console.log("persona 2:",persona2) // persona2:{ nombre: 'Pedro', edad: 30 }

//si queremos crear un clon:
//declaramos un objeto
let persona={
    "nombre": "Juan",
    "edad": 30
}

//clonamos el objeto
```

```
let persona3={ ... persona};
persona3.nombre="Luis";

console.log("persona:",persona) // persona:{ nombre: 'Juan', edad: 30 }
console.log("persona 3:",persona3) // persona2:{ nombre: 'Luis', edad: 30 }

//usar const para inmutabilidad
const nombre="juan perez"
nombre="maria uc" //error
```

Funciones

> **curso-node-funciones**
>
> Existen 2 tipos de funciones: funciones y funciones flecha
>
> ```
> //función
> function suma(a,b){
>  return a+b
> }
>
> //función flecha
> const resta=(a,b)=>{
>  return a-b
> }
>
> //funcion anonima
> console.log((a,b)=>{
>  return a*b
> })
> //función anonima
> console.log(function(a,b){
>  return a*b
> })
> ```
>
> ## Parámetros
>
> ```
> function tablaMultiplicar(tabla, hasta) {
>  for (let i = 0; i <= hasta; i++) {
>  console.log(tabla, "x", i, "=", tabla * i);
>  }
> }
> ```

```
// Ejecución
tablaMultiplicar(5, 3); // "tabla" valdrá 5 y "hasta" valdrá 3

// Parámetros x default
function saludo(nombre = "Visitante", mensaje = "Hola") {
  console.log(mensaje + ", " + nombre);
}

saludo(); // "Hola, Visitante"
saludo("Ana"); // "Hola, Ana"
saludo("Carlos", "Bienvenido"); // "Bienvenido, Carlos"
```

Parámetros rest

```
//Parámetros rest
function sumar( ... numeros) {
  return numeros.reduce((acumulador, valorActual) => acumulador +
valorActual, 0);
}

console.log(sumar(1, 2)); // 3
console.log(sumar(5, 10, 15)); // 30
```

Desestructurar Parámetros

```
// Sin desestructuración
function mostrarUsuario(usuario) {
  console.log(usuario.nombre);
  console.log(usuario.edad);
  console.log(usuario.ciudad);
}

// CON desestructuración
function mostrarUsuario({ nombre, edad, ciudad }) {
  console.log(nombre);
  console.log(edad);
  console.log(ciudad);
}

// Uso
```

```
const usuario = {
  nombre: "Ana",
  edad: 30,
  ciudad: "Madrid"
};

mostrarUsuario(usuario);
// Output: Ana, 30, Madrid
```

json

JSON (acrónimo de JavaScript Object Notation, 'notación de objeto de JavaScript') es un formato de texto sencillo para el intercambio de datos. Se trata de un subconjunto de la notación literal de objetos de JavaScript, aunque, debido a su amplia adopción como alternativa a XML, se considera un formato independiente del lenguaje.

```
//objeto en js
const persona={
  id:34,
  nombre:"Andrea",
  ap:"Flores",
  am:"Sanchez",
  sexo:"f",
  carrera:"ISC",
  edad:19,
  matricula:"244534",
  activo:'si'
}

console.log(persona)
//json en js
console.log(JSON.stringify(persona))
```

Objetos en js

> **curso-node-objetos**
>
> Los objetos en JavaScript son colecciones de propiedades, donde cada propiedad es un par clave-valor. La clave es una cadena (o símbolo) y el valor puede ser cualquier tipo de dato (número, cadena, función, otro objeto, etc.).
>
> ## Crear objetos

Forma 1 con Object:

```
var myCar = new Object();
myCar.make = "Ford";
myCar.model = "Mustang";
myCar.year = 1969;
```

Forma 2 usando iniciador de objeto:

```
var myCar = {
  make: "Ford",
  model: "Mustang",
  year: 1969,
};
```

Forma 3 con función constructora:

```
function Car(make, model, year) {
  this.make = make;
  this.model = model;
  this.year = year;
}
var mycar = new Car("Eagle", "Talon TSi", 1993);
```

Forma 4 con Object.create puedes heredar de otro objeto

```
const a = { a: 1 };
// acena de prototipos
// a ---> Object.prototype ---> null

const b = Object.create(a);
// cadena de prototipos
// b ---> a ---> Object.prototype ---> null
console.log(b.a); // 1 (inherited)

const c = Object.create(b);
// c ---> b ---> a ---> Object.prototype ---> null

const d = Object.create(null);
// d ---> null (d es un objeto que tiene null directamente como prototipo)
```

```
console.log(d.hasOwnProperty);
// undefined, porque d no hereda de Object.prototype
```

Otro ejemplo:

```
let padre = {
  saludo: function() {
    return "Hola";
  }
};

let hijo = Object.create(padre);
hijo.nombre = "Juan";

console.log(hijo.saludo()); // "Hola"
console.log(hijo.nombre); // "Juan"
```

Acceder a los atributos del objeto

Se accede a los elementos del objeto así:

```
myCar.make = "Ford";
myCar.model = "Mustang";
myCar.year = 1969;
//o asi
myCar["make"] = "Ford";
myCar["model"] = "Mustang";
myCar["year"] = 1969;
//Las propiedades no asignadas de un objeto son undefined (y no null).
//por ejemplo para el objeto anterior :
cosole.log(myCar.color); // undefined
```

desestructuración de objetos

```
const objeto = {
    propiedad: 'valor',
    otraPropiedad: 42
};

// Desesetructuración
const { propiedad } = objeto;
console.log(propiedad); // 'valor'
```

```
// Desestructuración con renombrado
const { propiedad: nuevoNombre, otraPropiedad: otroNuevoNombre } = objeto;
console.log(nuevoNombre); // 'valor'
console.log(otroNuevoNombre); // 42
console.log(propiedadVieja); // Error: propiedadVieja is not defined
```

ejemplo objetos

Un objeto persona:

```
const persona={
    id:0,
    nombre:"andres",
    carrera:"fisica",
    muestra:function(){
        console.log(`id:${this.id} nombre:${this.nombre}`)
    }
}

function fpersona(){
    this.id=0
    this.nombre="rosa"
    this.carrera="mate"
    this.muestra=function(){
        console.log(`id:${this.id} nombre:${this.nombre}`)
    }
}

persona3=new Object()
persona3.id=0
persona3.nombre="teresa"
persona3.carrera="algebra"
persona3.muestra=function(){
    console.log(`id:${this.id} nombre:${this.nombre}`)
}

persona.muestra()
const persona2=new fpersona()
persona2.muestra()
persona3.muestra()
```

Los objetos son asignados a otros objetos por referencia. Esto tiene el efecto que los vuelve mutables, Ejemplo:

```
//mutabilidad de objetos array
const array=[1,2,3,4,5]
const array2=array
array2[0]=45
console.log(array) //[ 45, 2, 3, 4, 5 ] , sin quererlo mutamos array
console.log(array2) //[ 45, 2, 3, 4, 5 ]
```

Clonar arreglos

```
const clon = [ ... arrayOriginal];
```

Clonar objetos

```
const original = { a: 1, b: 2 };
const clon = {  ... original };
```

Herencia

- `new Object()` o `{}` crea un objeto que hereda de `Object.prototype`.
- Para herencia de un objeto arbitrario, usa `Object.create(objetoPadre)`.
- Para herencia con funciones constructoras, establece el prototipo de la función constructora hija al prototipo del padre (usando `Object.create` y luego ajustando el `constructor`).
- Las clases de ES6 proporcionan una sintaxis más clara para la herencia.

Formas recomendadas de herencia

1. Object.create

```
const animal = {
  tipo: 'animal',
  respirar() {
    return 'respirando...';
  }
};

const perro = Object.create(animal); // Herencia real
```

```
perro.raza = 'labrador';
perro.ladrar = function() {
  return 'guau guau!';
};

console.log(Object.getPrototypeOf(perro) === animal); // true ✅
```

2. Clases ES6

```
class Animal {
  constructor(tipo) {
    this.tipo = tipo;
  }

  respirar() {
    return 'respirando...';
  }
}

class Perro extends Animal {
  constructor(raza) {
    super('mamifero');
    this.raza = raza;
  }

  ladrar() {
    return 'guau guau!';
  }
}

const miPerro = new Perro('labrador');
```

3. Funciones constructoras

```
function Animal(tipo) {
  this.tipo = tipo;
}

Animal.prototype.respirar = function() {
  return 'respirando...';
};

function Perro(raza) {
```

```
  Animal.call(this, 'mamifero');
  this.raza = raza;
}

// Herencia prototípica
Perro.prototype = Object.create(Animal.prototype);
Perro.prototype.constructor = Perro;

Perro.prototype.ladrar = function() {
  return 'guau guau!';
};
```

arreglos

curso-node-arreglos

declarando arreglos

```
// Usando el constructor Array
let arr1 = new Array(1, 2, 3);

// Usando la notación literal (recomendada)
let arr2 = [1, 2, 3];
```

acceso a elementos

```
let frutas = ['manzana', 'banana', 'cereza'];
console.log(frutas[0]); // 'manzana'
```

push pop

```
let arr = [1, 2, 3];
arr.push(4); // [1, 2, 3, 4]
arr.pop();   // [1, 2, 3]
arr.unshift(0); // [0, 1, 2, 3]
arr.shift(); // [1, 2, 3]
```

longitud

```
let arr = [1, 2, 3];
console.log(arr.length); // 3
```

inmutabilidad de arreglos

```
//inmutabilidad con operador spread
const array=[1,2,3,4,5]
const array2=[ ... array]
array[0]=45
console.log(array) //[ 45, 2, 3, 4, 5 ]
console.log(array2) //[ 1, 2, 3, 4, 5 ]
//map
const arreglo=[2,5,3,7,8,9]
//map --> para cada dato en el arreglo devuelve dato*2
const nvoArreglo=arreglo.map((dato)=>{
    return dato*2
})

//nvoArreglo es un nuevo arreglo donde su i-esimo elemento = i-esimo
elemento de arreglo * 2
```

map

Sintaxis

```
let nuevoArray = array.map(function(elemento, indice, arreglo) {
  // Procesar el elemento y devolver un nuevo valor
});
```

Ejemplo

```
let users = [
    {firstName : "Susan", lastName: "Steward"},
    {firstName : "Daniel", lastName: "Longbottom"},
    {firstName : "Jacob", lastName: "Black"}
  ];

  let userFullnames = users.map(function(element){
      return `<li>${element.firstName} ${element.lastName}</li>`;
```

```
  })
   const salida=userFullnames.join('\n')
  console.log(userFullnames);
  console.log(salida)
```

Ejemplo 2 map:

```
let users = [
  {firstName: "Susan", lastName: "Steward", edad: 34},
  {firstName: "Daniel", lastName: "Longbottom", edad: 27},
  {firstName: "Jacob", lastName: "Black", edad: 18}
];

// Obtener array con lastname y edad
let lastNamesAndAges = users.map(user => ({
  lastName: user.lastName,
  edad: user.edad
}));

console.log(lastNamesAndAges);
// Resultado:
// [
//   {lastName: "Steward", edad: 34},
//   {lastName: "Longbottom", edad: 27},
//   {lastName: "Black", edad: 18}
// ]
```

Filter

Sintaxis

```
const nuevoArray = array.filter(callback(elemento, indice, array) {
    // return condición (true para mantener, false para descartar)
});
```

Ejemplo

```
var arr = [
    { id: 15 },
    { id: -1 },
    { id: 0 },
    { id: 3 },
    { id: 12.2 },
```

```
    {},
    { id: null },
    { id: NaN },
    { id: "undefined" },
  ];

  var entradasInvalidas = 0;
  // Si el elemento tiene un atributo id, y su valor correspondiente es un
numero
  // Y no es el valor NaN, entonces es una entrada válida
  function filtrarPorID(obj) {
    if ("id" in obj && typeof obj.id === "number" && !isNaN(obj.id)) {
      return true;
    } else {
      entradasInvalidas++;
      return false;
    }
  }

  var arrPorID = arr.filter(filtrarPorID);

  console.log("Array Filtrado\n", arrPorID);
  // [{ id: 15 }, { id: -1 }, { id: 0 }, { id: 3 }, { id: 12.2 }]

  console.log("Número de Entradas Invalidas = ", entradasInvalidas);
  // 4
```

Ejemplo 2 filter:

```
let users = [
  {firstName: "Susan", lastName: "Steward", edad:34},
  {firstName: "Daniel", lastName: "Longbottom", edad:27},
  {firstName: "Jacob", lastName: "Black", edad:18}
];

let usuariosMenores20 = users.filter(user => user.edad < 20);

console.log(usuariosMenores20);
```

reduce

Sintaxis

```
array.reduce(
    callback(acumulador, valorActual[, indice[, array]]), valorInicial
)
```

Ejemplo

```
const array = [1, 2, 3, 4];
const valorInicial = 0;
const suma = array.reduce((acumulador, valorActual) => {
  return acumulador + valorActual;
}, valorInicial); // Resultado: 10
```

forEach

Sintaxis

```
array.forEach(function(elemento, indice, array) {
  // código a ejecutar por cada elemento
});
```

Ejemplo

```
const frutas = ['manzana', 'banana', 'naranja'];

frutas.forEach(function(fruta) {
  console.log(fruta);
});
// Salida:
// manzana
// banana
// naranja
```

funciones y this

En una funcion el valor de this está determinado por cómo se invoca a la función. No puede ser establecida mediante una asignación en tiempo de ejecución

El this en una funcion flecha es siempre el del contexto léxico que envuelve a la función.

En el contexto de ejecución global (fuera de cualquier función), this se refiere al objeto global, ya sea en modo estricto o no.

```
console.log(this.document === document); // true

// En los navegadores web, el objeto window también es un objeto global:
console.log(this === window); // true

this.a = 37;
console.log(window.a); // 37
```

Dentro de una función, el valor de this depende de cómo la función es llamada.

En una llamada simple:

```
function f1() {
  return this;
}
f1() === window; // objeto global

//
function f2() {
  "use strict"; // consultar modo estricto
  return this;
}
f2() === undefined;
```

Cuando es llamada con el operador dot(punto), el this toma el contexto del objeto que precede al punto

```
var o = {
  prop: 37,
  f: function () {
    return this.prop;
  },
};

console.log(o.f()); // logs 37
//como la funcion f es llamada con o. su contexto es el objeto o y this se
refiere al ambito del objeto o
```

Para ver que el comportamiento depende de como la funcion es llamada pruebe esto:

```
var o = { prop: 37 };

function independent() {
```

```
  return this.prop;
}

o.f = independent;

console.log(o.f()); // logs 37
console.log(independent()); //logs  undefined //prop esta indefinido

//el primer connsole.log llama a la funcon independet por medio  de o.f(), por
tanto el this es el de el contexto lexico de o
//en el segundo console.log la funcion independent se llama directamente,
//por  lo tanto no tiene un contexto lexico asociado y this esta indefinido
```

inmutabilidad de objetos

Suponga que tiene el objeto persona:

```
const persona={
    id:0,
    nombre:"andres",
    carrera:"fisica",
    muestra:function(){
        console.log(`id:${this.id} nombre:${this.nombre}`)
    }
}

//para garantizar la inmutabilidad debemos crear persona2 con el operador
spread (...),
//este operador hace una copia del objeto dado
const persona2={ ... persona,calificacion:90} //clonamos el objeto1 y le
agregamos un nuevo campo
persona.id=2
persona.nombre="andrea"
persona.muestra() //muestra el objeto original
persona2.muestra() //muestra un nuevo objeto con un campo calificacion
agregado
```

operador coalescencia nula

```
let name = obj.name ?? 'default name';
```

El operador ?? devolverá el lado izquierdo si no es nulo (es decir, nulo o indefinido), de lo contrario, devolverá el lado derecho. Entonces, en el ejemplo anterior, si obj.name no es nulo o

indefinido, el nombre se establecerá en obj.name . Pero si obj.name es nulo o indefinido, el nombre será 'nombre predeterminado'.

inmutabilidad en funciones que manejan objetos

```
const usuarios = [
    {
        id: 1,
        name: "brad gibson",
        email: "brad.gibson@example.com"
    },
    {
        id: 2,
        name: "Kuzey Karaduman",
        email: "kuzey.karaduman@example.com"
    },
    {
        id: 3,
        name: "Heinz-Werner Konrad",
        email: "heinz-werner.konrad@example.com"
    }
]
function agregaUsuario(usuarios){
    const newUser = {
        id:4,
        name: "Stephen Watkins",
        email: "stephen.watkings@example.com"
    }
    //usar push modifica el arreglo original
    //usuarios.push(newUser) // [{ id: 1}, {id:2}, {id: 3}, { id: 4} ]
    //para lograr la inmutabilidad usar spread
    const newUsuarios = [ ... usuarios, newUser ]
    retrun newUsuarios
}

const usuarios2=agregaUsuario(usuarios)
```

Para este mismo ejemplo si queremos insertar el usuario enmedio

```
const newUsuarios = [ ... usuarios.slice(0,1), newUser,  ... usuarios.slice(3)]
```

ordenar

```
//mutable
function compare(a, b) {
  if(a.id > b.id) {
    return -1
  }
    if(a.id < b.id) {
        return 1
    }
  return 0
}
usuarios.sort(compare); // [{ id: 3}, {id: 2}, {id: 1}]
//inmutable
const sorted = [ ... usuarios].sort(compare)
```

revertir el arreglo

```
//mutable
usuarios.revserse(); // [{ id: 1}, {id: 2}, {id: 3}]
//inmutable
const inverted = [ ... usuarios].reverse()
```

await

```
const leerDatos=async ()=>{
    const resp=await fetch('https://jsonplaceholder.typicode.com/posts')
    const data= await resp.json()
    return data
}

async function procesaDatos(){
    const data=await leerDatos()
    const dm=data.map(d=>{
        return(
            {
                "id":d.id,
                "title":d.title
            }
        )
    })
    console.log(dm)
}
```

```
procesaDatos()
```

solicitar datos get con fetch

```
const url='https://jsonplaceholder.typicode.com/posts'
fetch(url)
.then(function(response) {
    if(!response.ok){throw new Error("Error al recibir datos")}
    return response.json();
})
.then((data)=> {
    console.log('data = ', data);
    data.forEach((x)=>{
        ...
    })
})
.catch(function(err) {
    console.log("Error...");
    console.error(err);
});
```

//ejemplo solicitar datos con get usando async/await

```
const leerdatos=async(url)=>{
    try {
        const response=await fetch(url)
        if(!response.ok){throw new Error("Error en los datos")}
        const data=await response.json()
        console.log(data)
    } catch (error) {
        console.log("Eror:",error)
    }
}
leerdatos('https://jsonplaceholder.typicode.com/posts')
```

enviar formulario con fetch

```
//enviar datos de un formulario
const data = new FormData(document.getElementById('miformulario'));
fetch(url, {
    method: 'post',
```

```
    body: data
})
.then(function (response) {
    if (response.ok) {
        return response.text()
    } else {
        console.log("no se pudieron actualizar los datos")
        return Promise.reject("no se pudo insertar el dato")
    }
})
.then(resp => {
    console.log(resp);
})
.catch(function (err) {
    console.error(err);
})
```

En el código anterior, en el fetch, **no** se indica un **head** con la especificacion del formato en que se envian los datos, por que por default el fromato es 'Content-Type: application/x-www-form-urlencoded' , aunque tambien se podria poner así:

```
fetch(url, {
    method: 'post',
    headers: {
        "Content-Type": "application/x-www-form-urlencoded",
    }
    body: data
})
```

enviar json

```
//Tomar datos de un formulario y enviarlos en formato json
const frm=document.getElementById('miformulario')
//const frm=document.querySelector("#miformulario")
const data = new FormData(frm);
const datajson=Object.fromEntries(data)
fetch(url, {
    method: 'post',
    headers: {
        "Content-Type": "application/json",
    },
    body: JSON.stringify(datajson)
})
.then(function (response) {
```

```
    if (response.ok) {
        return response.text()
    } else {
        console.log("no se pudieron actualizar los datos")
        return Promise.reject("no se pudo insertar el dato")
    }
})
.then(resp => {
    console.log(resp);
});
.catch(function (err) {
    console.error(err);
})
```

En el ejemplo anterior como el formato en que deseamos enviar es json y no es el formato default, debemos especificar un encabezzado que indique que los datos los enviamos en formato json.

Ejemplo:

```
//enviar datos de un formulario, hay varias formas:
//1:tomar datos de un formulario
// const data = new FormData(document.getElementById('miformulario'));
//2:o en su defecto tomarlos directamente de un objeto
// const objData={"userId":"11","title":"mi titulo","body":"nuevo body"}
//3:o crear un FormData y asignar los datos direcctamente y a partir de ahi
consttruir un objeto
//usando fromEntries
const data = new FormData()
data.append("userId","11")
data.append("title","mi titulo")
data.append("body","nuvo body")
const objData=Object.fromEntries(data)
fetch('https://jsonplaceholder.typicode.com/posts', {
    method: 'POST',
    body: JSON.stringify(objData),
    headers: {
      'Content-type': 'application/json; charset=UTF-8',
    },
  })
.then((response) => response.json())
.then((json) => console.log(json));
```

Ejemplo con async/await

```
// const data = new FormData(document.getElementById('miformulario'));
const data = new FormData()
data.append("userId","11")
data.append("title","mi titulo")
data.append("body","nuvo body")
const objData=Object.fromEntries(data)
//
const envia=(url,objData)=>{
    try {
        const response=fetch(url, {
            method: 'POST',
            body: JSON.stringify(objData),
            headers: {
            'Content-type': 'application/json; charset=UTF-8',
            },
        })
        if(!response.ok){throw new Error("Erro en los datos")}
        const data=await response.json()
        console.log(data)
    } catch (error) {
        console.log("Eror:",error)
    }
}
envia('https://jsonplaceholder.typicode.com/posts',objData)
```

clases en js

Azucar sintactica para la creación de objetos en js

```
class Persona{
    #id
    #nombre
    constructor(id,nombre){
        this.#id=id
        this.#nombre=nombre
    }
    get nombre(){
        return this.#nombre
    }
    set nombre(nuevo){
        this.#nombre=nuevo
    }
}

const p=new Persona(4,"juana")
```

```
p.nombre="rosa"
console.log(p.nombre)
```

extends

```
class Animal {
  constructor(nombre) {
    this.nombre = nombre;
  }

  hablar() {
    console.log(this.nombre + " hace un ruido.");
  }
}

class Perro extends Animal {
  hablar() {
    console.log(this.nombre + " ladra.");
  }
}
```

También se pueden extender las clases tradicionales basadas en funciones:

```
function Animal(nombre) {
  this.nombre = nombre;
}
Animal.prototype.hablar = function () {
  console.log(this.nombre + "hace un ruido.");
};

class Perro extends Animal {
  hablar() {
    super.hablar();
    console.log(this.nombre + " ladra.");
  }
}

var p = new Perro("Mitzie");
p.hablar();
```

Fijarse que las clases no pueden extender objectos regulares (literales). Si se quiere heredar de un objecto regular, se debe user Object.setPrototypeOf():

super

```
class Gato {
  constructor(nombre) {
    this.nombre = nombre;
  }

  hablar() {
    console.log(this.nombre + ' hace ruido.');
  }
}

class Leon extends Gato {
  hablar() {
    super.hablar();
    console.log(this.nombre + ' maulla.');
  }
}
```

clases abstractas

```
class Persona{
  #id
  #nombre
  #correo
  constructor(id='',nombre='',correo=''){
    if(new.target === Persona){
      throw new Error("No se puede instancia esta clase")
    }
    this.id=id
    this.nombre=nombre
    this.correo=correo
  }
  muestra(){
    throw new Error("Este método debe ser implementado")
  }
}
class alumno extends Persona{
  muestra(){
    console.log(`id:${this.id}, nombre:${this.nombre}, correo:${this.correo}`)
  }
}
```

```
const p=new alumno(1,'ana','@mail')
p.muestra()
```

iteradores

```
function* fibonaci(){
  let x1=1;
  let x2=0;
  for(;true;){
      xs=x1+x2
      x1=x2
      x2=xs
      yield xs

  }
}
const f=fibonaci()
console.log(f.next())
console.log(f.next())
console.log(f.next())
console.log(f.next())
console.log(f.next())
```

salida:

{value: 1, done: false}
{value: 1, done: false}
{value: 2, done: false}
{value: 3, done: false}
{value: 5, done: false}

referencias

clases